

Stima della cardinalità nei flussi di dati distribuiti: merge e compatibilità semantica degli algoritmi di sketch

Seminario magistrale

Daniele Ferroli

Università degli Studi di Udine

A.A. 2024–2025

- ① Problema del conteggio degli elementi distinti
- ② Fondamenti teorici
- ③ Analisi dello stato dell'arte
- ④ Implementazione: architettura, hash, dataset e validazione
- ⑤ Risultati principali
- ⑥ Conclusioni, limiti e sviluppi futuri

Problema e contributo del lavoro

- Obiettivo: stimare quanti identificatori distinti compaiono in un flusso continuo senza conservare tutto l'input.
- Applicazioni tipiche: monitoraggio di rete, telemetria, osservabilità, analisi di log, ottimizzazione di interrogazioni.
- Nei sistemi distribuiti non basta stimare bene in locale: serve anche combinare sketch prodotti su nodi o partizioni diverse.
- La tesi studia il comportamento reale degli sketch e del loro merge, senza proporre un nuovo stimatore teorico.

Contributo principale

Implementazione uniforme, hash configurabili, framework riproducibile e classificazione dei casi di merge: `valid`, `recoverable`, `invalid`.

Modello minimo: stream, F_0 e sketch

- Il flusso è una sequenza ordinata $S = \langle x_1, x_2, \dots, x_s \rangle$.
- Modello *insertion-only*: ogni aggiornamento aggiunge occorrenze, senza rimozioni.
- Gli algoritmi lavorano in uno o pochi passaggi, con memoria sublineare.
- Nel framework esiste anche un riferimento esatto interno, NaiveCounting, usato per test e controlli.

$$F_0 = |\{a \in \mathcal{U} \mid f(a) > 0\}|$$

Interpretazione

F_0 è la cardinalità del supporto del vettore delle frequenze, cioè il numero di elementi distinti osservati.

Vincolo di fondo

Il conteggio esatto non scala bene quando il flusso cresce; da qui nasce l'uso di sketch probabilistici.

Evoluzione storica dello stato dell'arte

Metodo	Idea dominante	Ruolo nella tesi
Probabilistic Counting	Bitmap singola che osserva eventi rari sull'hash	Primo punto della linea storica e benchmark implementato
PCSA	Mediazione stocastica su più bitmap virtuali	Passo intermedio ricostruito nello stato dell'arte
LogLog	Registri e media geometrica dei massimi di rango	Benchmark implementato
HyperLogLog	Media armonica e correzioni di regime	Benchmark implementato e riferimento vicino a HLL++
HyperLogLog++	Bias correction + stato sparso/denso	Riferimento pratico principale per gli esperimenti di merge

Lo stato dell'arte richiama anche Bloom filter e Count-Min Sketch per mostrare che lo sketching si estende oltre il solo conteggio dei distinti.

Algoritmi implementati nel framework

ID	Metodo	Ruolo
naive	NaiveCounting	riferimento esatto interno
pc	Probabilistic Counting	benchmark pubblico
ll	LogLog	benchmark pubblico
hll	HyperLogLog	benchmark pubblico
hllpp	HyperLogLog++	benchmark pubblico

- Interfaccia comune con `process`, `count`, `merge` e `reset`.
- Hash iniettata nel costruttore: fa parte del contesto dello sketch.
- HLL++ aggiunge stato sparso/denso, tabelle di bias e riduzione alla precisione minima.

Sketch, hash e mergeabilità

- Uno sketch mantiene una sintesi compatta del flusso.
- Negli sketch count-distinct l'hash determina come il flusso viene codificato nello stato.
- Il merge corretto deve preservare il significato dello stato, non solo la sua forma.

$$\hat{\theta}(\mathcal{K}(S_1) \oplus \mathcal{K}(S_2)) \approx \hat{\theta}(\mathcal{K}(S_1 \cdot S_2))$$

$$(\mathcal{K}(S_1) \oplus \mathcal{K}(S_2))[j] = \max\{\mathcal{K}(S_1)[j], \mathcal{K}(S_2)[j]\}$$

Punto critico

Per gli sketch HLL-like il merge a massimi è corretto solo con parametri compatibili e stessa semantica dell'hash.

Compatibilità semantica del merge

valid	recoverable	invalid
Il merge è corretto direttamente. Caso tipico: HLL++ con stessa precisione, stessa funzione di hash e stesso seed.	Il merge non è corretto subito, ma lo diventa dopo una normalizzazione esplicita. Caso tipico: HLL++ con precisioni diverse ma stessa semantica dell'hash.	In questo quadro non esiste un merge semanticamente corretto. Caso tipico: funzioni di hash o seed diversi.

- Strategia operativa distinta dalla compatibilità: `direct`, `reject`, `reduce_then_merge`.
- `unsafe_naive_merge` viene mantenuto solo come controllo negativo sperimentale.

Architettura generale del sistema

Algoritmi

Sketch concreti e catalogo degli identificatori: h11pp, h11, 11, pc.

Supporto

Dataset binario, lettura per partizione, funzioni di hash e metadati di esecuzione.

Valutazione

CLI, job di esperimento, framework di simulazione, metriche e scrittura dei risultati CSV.

Scelta progettuale

Algoritmi, hash, dataset e metriche sono separati per rendere gli esperimenti riproducibili ed estendibili.

Hash, riproducibilità e scelte progettuali

Livello hash

- Interfaccia comune: `hash64`, `hash32`, `name`.
- Funzioni supportate: `splitmix64`, `xxhash64`, `murmurhash3`, `siphash24`.
- L'hash è parte della semantica dello sketch, non un dettaglio di implementazione.

Riproducibilità

- Il seed del dataset è nell'header binario.
- Nel merge eterogeneo i due lati possono avere seed/hash separati.
- I metadati del dataset sono derivati automaticamente.

Dataset prefix_constant_rho e validazione

- Dataset sintetico controllato con $n = 10^7$, $p = 50$, seed = 21041998.

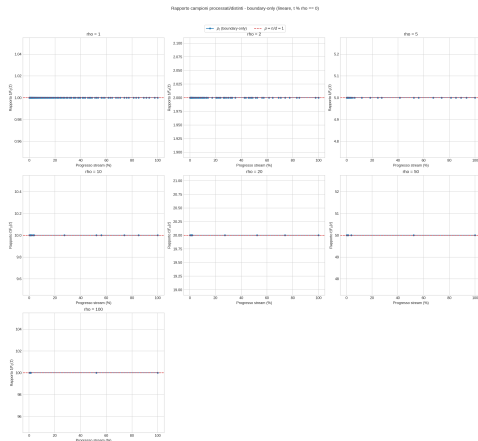
- Rapporto controllato:

$$\rho = \frac{n}{d} \in \{100, 50, 20, 10, 5, 2, 1\}.$$

- Una nuova prima occorrenza per ogni blocco di lunghezza ρ :

$$F_0(j \cdot \rho) = j.$$

- Il bitset di verità permette di ricostruire $F_0(t)$ sui prefissi.



La validazione empirica conferma il comportamento atteso di $\rho_t = t/F_0(t)$ ai bordi di blocco.

Test dedicati coprono algoritmi, merge, framework, schema CSV e generatore del dataset.

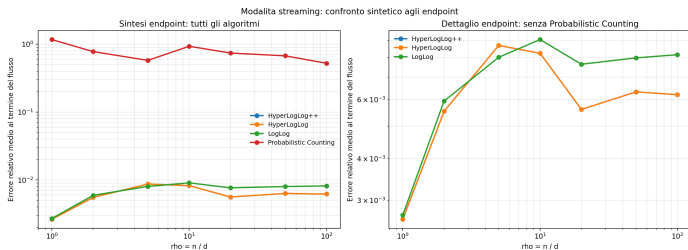
Disegno sperimentale

- Ambiente di test: AMD Ryzen 7 9700X, 32 GB DDR5, Clang 22.1.1, CMake 4.3.0.
- Tre modalità di valutazione:
 - **streaming**: andamento della stima lungo i prefissi;
 - **merge omogeneo**: confronto tra merge e riferimento seriale;
 - **merge eterogeneo**: casi *valid*, *recoverable*, *invalid*.
- Due viste complementari in streaming:
 - caso A: stima in funzione della cardinalità reale del prefisso;
 - caso B: stima in funzione di $\rho_t = t/F_0(t)$, con

$$F_0^* \in \{1000, 2000, 5000, 10000, 20000, 50000, 100000\}.$$

- Metriche: stima media, dispersione, errore relativo medio ai punti finali.
- Gli esperimenti di merge sono concentrati su HyperLogLog++.

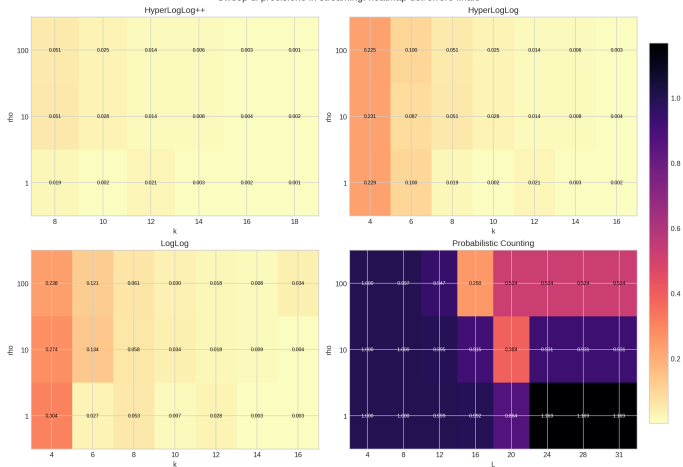
Risultato 1: comportamento degli algoritmi in streaming



- Su $F_0(t)$, HLL e HLL++ sono quasi sovrapposti: errore medio circa 0.00618.
- Su ρ_t , HLL++ è il migliore: $3.95 \cdot 10^{-4}$, contro $2.08 \cdot 10^{-3}$ di HLL.
- LogLog è più sensibile e nella seconda vista raggiunge circa 1.455.
- Probabilistic Counting resta instabile: circa 0.770 nella prima vista e 0.346 nella seconda.

Risultato 2: l'aumento di precisione favorisce soprattutto HLL++

Sweep di precisione in streaming: heatmap dell'errore finale



Miglior parametro osservato

- HLL++: $k = 18$ per tutti i valori di ρ .
 - HLL: $k = 16$ in media; con $\rho = 1$, $k = 10$.
 - LogLog: $k = 14$ in media; con $\rho = 10$, $k = 16$.
 - PC: $L = 20$ in media; con $\rho = 100$, $L = 16$.
-
- HLL++ è il metodo con andamento più regolare e prevedibile.
 - Gli altri metodi dipendono di più dal parametro.

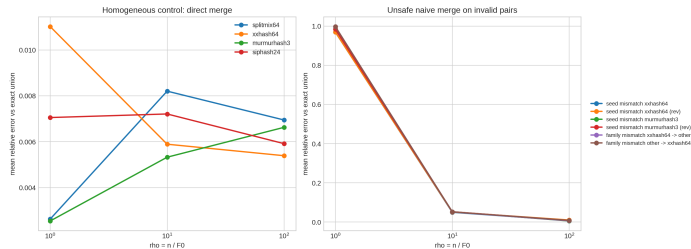
Risultato 3: il merge omogeneo è il controllo del caso valid

- Configurazione: HyperLogLog++, $\rho \in \{1, 10, 100\}$, $k \in \{10, 14, 18\}$, quattro funzioni di hash.
- Condizione di omogeneità: stessa funzione di hash, stesso seed e stessi parametri sui due lati.
- Totale osservato: 36 configurazioni e 900 coppie di merge.

$$\Delta_{abs} = 0, \quad \Delta_{rel} = 0$$

ρ	d	Coppie totali	Coppie con delta $\neq 0$	Max Δ_{rel}
1	10^7	300	0	0
10	10^6	300	0	0
100	10^5	300	0	0

Risultato 4: hash mismatch = caso invalid

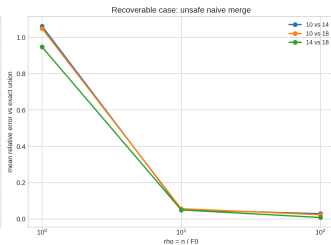
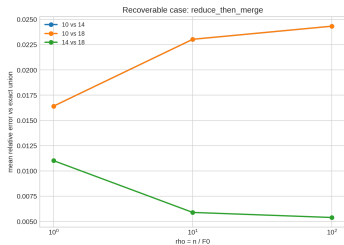


- Seed mismatch: stessa hash xxhash64 o murmurhash3, ma seed diversi.
- Family mismatch: xxhash64 contro splitmix64, murmurhash3 o siphash24; anche direzione inversa.
- Controllo omogeneo locale: errore medio = 0.006233.
- Casi invalid: errore medio tra 0.342374 e 0.352802.
- Errore massimo osservato: 1.009634.

Conclusione

Se cambia la semantica dell'hash, il merge forzato perde significato. In questo contesto il caso va rifiutato.

Risultato 5: precision mismatch = caso recoverable



- Hash xxhash64, stesso seed.
- Coppie: (10, 14), (10, 18), (14, 18) e versioni invertite.
- reject: nessuna stima di merge, riferimento seriale = 0.016651.

Strategia	$\bar{\epsilon}_{merge}$
unsafe_naive_merge	0.363882
reduce_then_merge	0.016651

- reduce_then_merge coincide con il riferimento corretto.
- Il merge forzato raggiunge massimo 1.061378 e degrado medio 32.37.

Sintesi dei risultati e contributi

Risultati empirici

- HLL++ è il metodo più stabile nel dominio osservato.
- Il merge omogeneo coincide col riferimento seriale.
- L'eterogeneità dell'hash rompe la correttezza del merge.
- Il mismatch di precisione è recuperabile solo tramite normalizzazione esplicita.

Contributi del lavoro

- Implementazione uniforme di più algoritmi count-distinct.
- Interfaccia comune per hash, dataset, metriche e serializzazione.
- Framework riutilizzabile per streaming e merge distribuito.
- Lessico operativo `valid/recoverable/invalid`.

Take-away

Negli sketch count-distinct distribuiti, hash e parametri fanno parte della semantica dello sketch: senza compatibilità semantica il merge non è un'operazione neutra.

Limiti del lavoro

- Dataset soprattutto sintetico e controllato
- Studio del merge concentrato soprattutto su casi pairwise
- Analisi eterogenea limitata a hash mismatch e precision mismatch
- Parte di merge sviluppata soprattutto su HLL++

Direzioni future

- Dati reali e integrazione con sorgenti come Apache Kafka
- Topologie distribuite più ampie e scelta del punto ottimale di riduzione alla precisione minima
- Altri assi di eterogeneità semanticamente chiari
- Algoritmi più recenti come UltraLogLog, SetSketch, HyperLogLogLog
- Estensione del framework a Bloom filter, Count-Min Sketch e ad altri costi di merge

Grazie per l'attenzione.